

Program Reasoning

9. Automated Program Verification

Kihong Heo



Towards Fully Automated Verification

- The assumption so far: a user provides inductive invariants
- Fully automated verification: combined with automated invariant generation methods
- Example:
 - Program analysis [CS524]: automatic, terminating, but may not be exact
 - Machine learning: automatic? terminating? exact?
- This lecture: verification via constraint solving
 - With Constrained Horn Clause (CHC)
 - Using SMT solvers

Horn Clause

- Clause: a disjunction of literals
 - E.g., $p \vee \neg q \vee \neg r$
- Horn clause: a clause with at most one positive literal
 - E.g., $\neg p \vee \neg q \vee r$ which is equivalent to $p \wedge q \rightarrow r$
- Horn clause logic: the basis of logic programming languages such as Prolog and Datalog
- Why Horn clause? Efficiency
 - Propositional Horn clause (HORNSAT): linear time
 - First-order Horn clause (e.g., Prolog): undecidable but efficient
- More details: CS402 (Introduction to Logic for Computer Science)



A. Horn

Constrained Horn Clause (CHC)

- A fragment of first-order logic

$$\underbrace{\forall x. \varphi}_{\text{Constraint}} \wedge \underbrace{p_1(X_1) \wedge \cdots \wedge p_n(X_n)}_{\text{Predicates}} \rightarrow h(X)$$

- φ : a constraint in a background theory T
 - E.g., $x + 1 = 2$ (Peano arithmetic)

Example

- Are the CHC formulas satisfiable? If so, what is P ?

$$P(0)$$

$$\forall x, x'. P(x) \wedge x < 10 \wedge x' = x + 1 \rightarrow P(x')$$

$$\forall x. P(x) \wedge x > 11 \rightarrow \textit{false}$$

$$\forall x. x \leq 0 \rightarrow P(x)$$

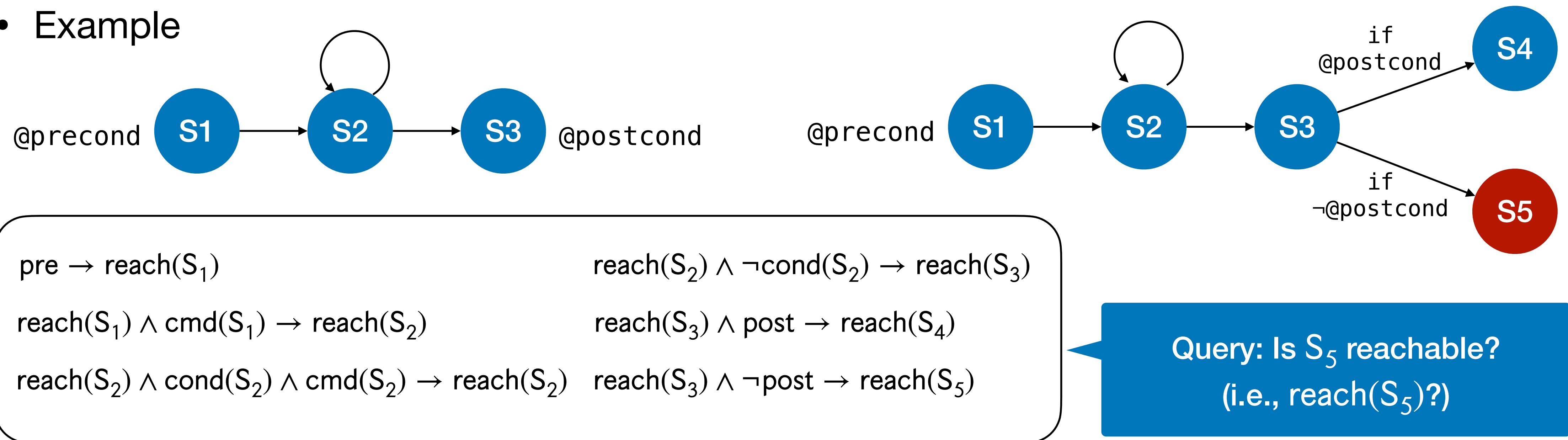
$$\forall x, x'. P(x) \wedge x < 5 \wedge x' = x + 1 \rightarrow P(x')$$

$$\forall x. P(x) \wedge x > 5 \rightarrow \textit{false}$$

Program Verification via CHC

- Given a program and a specification, generate verification conditions using CHC
- Check the satisfiability of the CHC formula using SMT solvers
- Idea: partial correctness check as a reachability problem

- Example



Language

- Program = control flow graph
- Node = basic block = list of commands (end with jump)

$$\begin{aligned} C &\rightarrow \text{skip} \mid x := E \mid x := \text{input}() \mid \text{br } B \ l_1 \ l_2 \\ &\quad \mid \text{goto } l \mid \text{assume}(E) \mid \text{assert}(E) \\ E &\rightarrow n \mid x \mid E + E \mid E - E \mid E \times E \mid E / E \\ B &\rightarrow \text{true} \mid \text{false} \mid E < E \mid E = E \mid \neg B \end{aligned}$$

- Example:

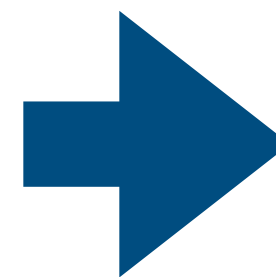
```
// P1
Entry:
  x := input()
  assume(x > 1)
  assert(x == 0)
```

```
// P2
Entry:
  x := input()
  y := x - 1
  br x / 2 != 0 L1 L2
L1:
  assert(y != 0)
L2:
  skip
```

Specification

- Annotated in programs using assertions
- Checking an assertion = checking a reachability
 - Assertion is false = error state is reachable
- Example

Entry:
x := input()
y := x - 1
assert(y != 0)



Entry:
x := input()
y := x - 1
br y != 0 L1 L2
L1:
skip
L2:
assert false

State

- A predicate parameterized by values of variables defined so far
 - One relation per basic block
- Example

```
Entry:
  x := input()
  y := x - 1
  br y != 0 L1 L2
L1:
  skip
L2:
  assert false
```

Relations: Entry, L1(x, y), L2(x, y)

Reachable states:

Entry,
L1(2, 1), L1(3,2), L1(4, 3), ...
L2(1, 0)

Verification Condition

- CHC formula: the relationship among all nodes + unreachability of the error node
- Loop invariants will be computed by the underlying solver (But not always! Why?)
- Example

Entry:
x := input()
y := x - 1
assert(y != 0)



Entry:
x := input()
y := x - 1
br y != 0 L1 L2
L1:
skip
L2:
assert false

The condition is SATISFIABLE
iff
the program is correct

Entry

$\forall x, y. \text{Entry} \wedge y = x - 1 \wedge y \neq 0 \rightarrow L_1(x, y)$

$\forall x, y. \text{Entry} \wedge y = x - 1 \wedge y = 0 \rightarrow L_2(x, y)$

$\forall x, y. L_2(x, y) \rightarrow \text{false}$

Example (1)

```
x := input();
assume(x < 10);
while(x < 10) {
  x++;
}
assert(x == 10);
```



Entry:

```
x0 := input()
assume(x0 < 10)
goto Cond
```

Cond:

```
x1 := φ [x0, Entry] [x2, Body]
br (x1 < 10) Body End
```

Body:

```
x2 := x1 + 1
goto Cond
```

End:

```
br (x1 = 10) Then Else
```

Then

```
skip
```

Else:

```
assert false
```

The condition is SATISFIABLE
iff
the program is correct

Entry

$\forall x. \text{Entry} \wedge x < 10 \rightarrow \text{Cond}(x)$

$\forall x. \text{Cond}(x) \wedge x < 10 \rightarrow \text{Body}(x)$

$\forall x. \text{Cond}(x) \wedge x \geq 10 \rightarrow \text{End}(x)$

$\forall x, x'. \text{Body}(x) \wedge x' = x + 1 \rightarrow \text{Cond}(x')$

$\forall x. \text{End}(x) \wedge x = 10 \rightarrow \text{Then}(x)$

$\forall x. \text{End}(x) \wedge x \neq 10 \rightarrow \text{Else}(x)$

$\forall x. \text{Else}(x) \rightarrow \text{false}$

Reachable states:

Entry	: T
Cond(10), Cond(9), Cond(8), Cond(7), ...,	: $x \leq 10$
Body(9), Body(8), Body(7), ...,	: $x \leq 9$
End(10)	: $x = 10$
Then(10)	: $x = 10$

$\{ T \} \text{Entry} \{ x \leq 10 \}$
 $\{ x \leq 10 \} \text{Cond}; \text{Body} \{ x = 10 \}$

Example (2)

- In practice, typically solve the incorrectness rather than correctness for efficiency
 - Query the reachability to an error state from the initial state
 - E.g., in your homework

The condition is SATISFIABLE
iff
the program is correct

Entry

$\forall x. \text{Entry} \wedge x < 10 \rightarrow \text{Cond}(x)$

$\forall x. \text{Cond}(x) \wedge x < 10 \rightarrow \text{Body}(x)$

$\forall x. \text{Cond}(x) \wedge x \geq 10 \rightarrow \text{End}(x)$

$\forall x, x'. \text{Body}(x) \wedge x' = x + 1 \rightarrow \text{Cond}(x')$

$\forall x. \text{End}(x) \wedge x = 10 \rightarrow \text{Then}(x)$

$\forall x. \text{End}(x) \wedge x \neq 10 \rightarrow \text{Else}(x)$

$\forall x. \text{Else}(x) \rightarrow \text{false}$

The condition is SATISFIABLE
iff
the program is **incorrect**

Entry

$\forall x. \text{Entry} \wedge x < 10 \rightarrow \text{Cond}(x)$

$\forall x. \text{Cond}(x) \wedge x < 10 \rightarrow \text{Body}(x)$

$\forall x. \text{Cond}(x) \wedge x \geq 10 \rightarrow \text{End}(x)$

$\forall x, x'. \text{Body}(x) \wedge x' = x + 1 \rightarrow \text{Cond}(x')$

$\forall x. \text{End}(x) \wedge x = 10 \rightarrow \text{Then}(x)$

$\forall x. \text{End}(x) \wedge x \neq 10 \rightarrow \text{Else}(x)$

Query(Else)

Summary

- Automated program verification for partial correctness
 - Equivalently, checking reachability of error states
- Constrained Horn clause: a fragment of FOL
- Program verification using CHC
 - Verification condition = unreachability of error states
- Automatically solved by theorem provers